CHANDIGARH UNIVERSITY ONLINE

Discover. Learn. Empower.

MCA PROJECT REPORT

Academic Portal

A Web-Based System for Course Management, Appointment Scheduling, and Student Support

Submitted in Partial Fulfillment of the Requirements for the Degree of

Master of Computer Applications (MCA)

Student Name
[Your Name]
Roll Number
[Your Roll Number]
Guide Name
[Guide Name]
Department
Master of Computer Applications
Institution
[Your Institute Name]
Submission Date
May 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled:

"ACADEMIC PORTAL"

submitted to [Your Institute Name] for the partial fulfillment of the requirements for the degree of Master of Computer Applications (MCA) is a record of bona fide work carried out by [Your Name] (Roll No: [Roll Number]) under the guidance of [Guide Name], Department of Computer Applications.

The project has been completed satisfactorily and is recommended for evaluation.

Signature of Guide: _____ Date: _____

[Guide Name]

Designation: [Designation]

Department of Computer Applications

Signature of Head of Department: _____ Date: _____

[HOD Name]

Head of Department, Computer Applications

DECLARATION BY STUDENT

I, [Your Name], student of Master of Computer Applications (MCA), hereby declare that the project entitled "Academic Portal" submitted to [Your Institute Name] is my original work and has not been submitted earlier to any university or institution for the award of any degree or diploma.

All the information, data, and results presented in this report are true and accurate to the best of my knowledge. I have duly acknowledged all sources of information used in this project report. I have not fabricated or misrepresented any data or results.

I also declare that this work conforms to academic integrity standards and complies with the anti-plagiarism policy of the university.

Student Signature: _____ Date: _____

[Your Name]

Roll No: [Roll Number]

MCA — [Year/Semester]

Place: _____

ACKNOWLEDGEMENT

I express my sincere and heartfelt gratitude to my project guide, [Guide Name], for their invaluable guidance, unwavering support, constructive feedback, and continuous encouragement throughout the development of this project. Their expertise and patience have been instrumental in shaping this work.

I am deeply thankful to the Head of the Department of Computer Applications, [HOD Name], for providing the necessary resources, infrastructure, and a conducive academic environment that made this project possible.

I would also like to extend my thanks to all the faculty members of the department for their academic support and for imparting the knowledge that formed the foundation of this project. Their teachings during the MCA programme have been essential to my understanding of software development and system design.

My deepest appreciation goes to my family and friends for their moral support, patience, and encouragement during the entire duration of this project. Their belief in my abilities kept me motivated during challenging times.

Finally, I am grateful to all the open-source contributors whose libraries and tools — including ASP.NET Core, Bootstrap, AdminLTE, and SweetAlert2 — formed the technological backbone of this project.

Signature: _____ Date: _____

[Your Name]

ABSTRACT

This project presents the design and implementation of an "Academic Portal" — a comprehensive, web-based information system developed to streamline the academic operations of educational institutions. The portal integrates essential functionalities including user authentication with role-based access control, course management, student enrollment, faculty profile management, appointment scheduling, feedback collection, institutional policy management, and a lightweight rule-based conversational chatbot to assist users with queries.

The system is developed using ASP.NET Core MVC targeting the .NET 8 framework, with Razor views for server-side rendering, ADO.NET with Microsoft.Data.SqlClient for database access, and Microsoft SQL Server as the backend database. The frontend employs Bootstrap and AdminLTE for a consistent, responsive user interface, jQuery for dynamic interactions, and SweetAlert2 for enhanced user notifications.

The development methodology followed an Agile-inspired incremental approach, with modules built progressively: Authentication → Courses → Appointments → Chatbot → Feedback → Policies. The appointment booking module implements transactional database operations using SqlTransaction to prevent race conditions and double-booking scenarios. Security measures including parameterized SQL queries and antiforgery token validation are applied throughout the application.

The report thoroughly documents the software development lifecycle: requirements analysis, feasibility study, UML-based system design, module-wise implementation with code explanations, comprehensive test cases, performance observations, and a future enhancement roadmap. The system demonstrates proficiency in full-stack web development, database design, system architecture, and academic software engineering practices meeting MCA programme evaluation criteria.

Keywords: Academic Portal, ASP.NET Core MVC, Role-Based Access Control, Appointment Scheduling, Chatbot, SQL Server, ADO.NET, Web Application.

TABLE OF CONTENTS

Bonafide Certificate ..... **ii**
Declaration ..... **iii**
Acknowledgement ..... **iv**
Abstract ..... **v**
List of Figures ..... **viii**
List of Tables ..... **ix**
List of Abbreviations ..... **x**
Chapter 1: Introduction **1**
1.1 Background of the Project 1
1.2 Problem Statement 2
1.3 Objectives of the System 3
1.4 Scope of the Project 4
1.5 Overview of Existing Systems 5
1.6 Proposed System Overview 6
1.7 Technologies Used 7
Chapter 2: Literature Review / System Study **9**
2.1 Review of Similar Systems 9
2.2 Comparative Analysis 11
2.3 Software Development Models 13
2.4 Frameworks and Libraries 14
2.5 Research Gap 16
Chapter 3: System Analysis **18**
3.1 Functional Requirements 18
3.2 Non-Functional Requirements 20
3.3 User Requirements 21
3.4 Feasibility Study 22
3.5 System Architecture 24
3.6 Data Flow Diagrams (DFD) 25
3.7 Use Case Diagrams 27
Chapter 4: System Design **29**
4.1 UML Diagrams 29
4.2 Entity-Relationship (ER) Diagram 34
4.3 Database Schema and Table Structures 36
4.4 API / Endpoint Design 40
4.5 UI/UX Wireframes 42

Chapter 5: System Implementation.....45
5.1 Development Environment.....45
5.2 Tools and Technologies.....46
5.3 Hardware and Software Requirements.....48
5.4 Module-wise Explanation.....49
5.5 Key Algorithms.....57
5.6 Security Features.....60
5.7 Screenshots of Application.....62
Chapter 6: Testing.....66
6.1 Testing Strategy.....66
6.2 Unit Testing.....67
6.3 Integration Testing.....69
6.4 System Testing.....70
6.5 Test Cases Table.....71
6.6 Bug Reports and Resolutions.....73
Chapter 7: Results and Discussion.....75
7.1 Functionality Achieved.....75
7.2 Performance Evaluation.....76
7.3 Comparison with Existing Systems.....77
7.4 Limitations.....78
Chapter 8: Conclusion and Future Enhancements.....80
8.1 Conclusion.....80
8.2 Key Achievements and Learning Outcomes.....81
8.3 Future Scope and Enhancements.....82
Chapter 9: References.....85
Chapter 10: Appendices.....88
Appendix A: File and Module Map.....88
Appendix B: Database Scripts.....89
Appendix C: Installation Guide.....91
Appendix D: User Manual.....92

LIST OF FIGURES

Figure 4.1 — High-Level System Architecture Diagram.....29
Figure 4.2 — Use Case Diagram — Admin Role.....30
Figure 4.3 — Use Case Diagram — Faculty Role.....31
Figure 4.4 — Use Case Diagram — Student Role.....32
Figure 4.5 — Class Diagram — Core Entities.....33
Figure 4.6 — Sequence Diagram — Appointment Booking.....34
Figure 4.7 — Activity Diagram — User Login Flow.....35
Figure 4.8 — Entity-Relationship (ER) Diagram.....36
Figure 4.9 — UI Wireframe — Student Dashboard.....42
Figure 4.10 — UI Wireframe — Appointment Booking Page.....43
Figure 3.1 — DFD Level 0 — Context Diagram.....25
Figure 3.2 — DFD Level 1 — Booking Flow.....26
Figure 5.1 — Login Page Screenshot.....62
Figure 5.2 — Admin Dashboard Screenshot.....63
Figure 5.3 — Student Dashboard Screenshot.....63
Figure 5.4 — Appointment Booking Page Screenshot.....64
Figure 5.5 — Chatbot Widget Screenshot.....65
Figure 5.6 — Feedback Submission Form Screenshot.....65
Figure 7.1 — Page Load Performance Chart.....76
Figure 7.2 — Comparison Chart — Academic Portal vs Existing Systems.....77

LIST OF TABLES

Table 2.1 — Comparative Analysis of Academic Systems....11
Table 3.1 — Functional Requirements....18
Table 3.2 — Non-Functional Requirements....20
Table 3.3 — Feasibility Study Summary....22
Table 4.1 — Users Table Structure....36
Table 4.2 — Courses Table Structure....37
Table 4.3 — AppointmentSlots Table Structure....37
Table 4.4 — Appointments Table Structure....38
Table 4.5 — Feedback Table Structure....39
Table 4.6 — Policies Table Structure....39
Table 4.7 — API Endpoints Summary....40
Table 5.1 — Hardware Requirements....48
Table 5.2 — Software Requirements....48
Table 6.1 — Unit Test Cases....67
Table 6.2 — Integration Test Cases....69
Table 6.3 — System Test Cases....70
Table 6.4 — Bug Report Log....73

LIST OF ABBREVIATIONS

Abbreviation	Full Form
MCA	Master of Computer Applications
MVC	Model–View–Controller
API	Application Programming Interface
SQL	Structured Query Language
DB	Database
UI	User Interface
UX	User Experience
ER	Entity-Relationship
DFD	Data Flow Diagram
UML	Unified Modeling Language
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
OS	Operating System
RAM	Random Access Memory
SHA	Secure Hash Algorithm
JWT	JSON Web Token
APA	American Psychological Association
SDLC	Software Development Life Cycle
LMS	Learning Management System
SSO	Single Sign-On
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JS	JavaScript

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

In the modern digital era, educational institutions are increasingly adopting information technology solutions to manage their academic processes efficiently. The complexity of managing student data, course information, faculty schedules, and administrative communications has grown substantially as institutions scale. Traditional manual processes — including paper-based enrollment, handwritten appointment registers, and physical notice boards for policy dissemination — are no longer adequate to meet the demands of contemporary academic environments.

The proliferation of web technologies, particularly model-view-controller (MVC) frameworks built on server-side platforms such as ASP.NET Core, has made it feasible to develop robust, multi-tier web applications that can automate and centralize academic workflows. These applications can simultaneously serve multiple user roles — administrators, faculty, and students — with personalized interfaces and appropriate access privileges.

Academic portals have become a standard feature in universities worldwide. However, the majority of commercially available learning management systems (LMS) — such as Moodle, Blackboard, or Canvas — are feature-heavy, difficult to customize for specific institutional requirements, and often carry significant licensing costs. There exists a genuine need for lightweight, tailored solutions that offer core academic functionalities without the overhead of enterprise-scale platforms.

The Academic Portal developed as part of this MCA project addresses this gap by providing a streamlined, role-based web application that supports the core academic operations of a medium-sized department: user and course management, transactional appointment booking between students and faculty, institutional policy publication, student feedback submission, and a simple chatbot assistant for user guidance.

1.2 Problem Statement

Academic departments in many institutions continue to rely on fragmented and inefficient methods for managing student–faculty interactions and administrative information. The specific problems identified during requirement analysis are:

- Appointment Scheduling: Students must physically approach faculty or administrative offices to schedule appointments, resulting in lost time, scheduling conflicts, and unrecorded bookings.
- Course Enrollment: Enrollment processes are either paper-based or managed through disconnected spreadsheets, making it difficult to track enrollments, manage capacity, or generate reports.
- Information Dissemination: Institutional policies, examination schedules, and academic notices are communicated through notice boards or email, leading to missed updates and inconsistent access.
- Feedback Collection: Student feedback for courses and faculty is either informal or collected through paper forms, making aggregation, analysis, and follow-up difficult.
- User Assistance: Students frequently have repetitive queries regarding booking processes, course offerings, and institutional policies. Manual query handling consumes significant administrative time.

These challenges collectively result in administrative inefficiencies, student dissatisfaction, and poor utilization of institutional resources. The proposed Academic Portal directly addresses these problems through a unified, web-based solution.

1.3 Objectives of the System

The primary objectives of the Academic Portal are as follows:

1. Role-Based Access Control: Implement a secure authentication system supporting three distinct roles — Admin, Faculty, and Student — with role-specific dashboards, menus, and data access permissions.
2. User Management: Allow administrators to create, edit, and manage user accounts for faculty and students, including role assignment and profile maintenance.

3. Course Management: Enable administrators to perform CRUD operations on courses; allow students to browse the course catalogue and enroll in available courses with automatic enrollment tracking.
4. Appointment Scheduling: Provide faculty with the ability to define available time slots and students with the ability to book, view, and cancel appointments using transactional database operations to prevent double-booking race conditions.
5. Policy Management: Allow administrators to publish and update institutional policies that are accessible to all authenticated users through a searchable interface.
6. Feedback System: Enable students to submit ratings and comments for courses and faculty, with automatic sentiment tagging to assist in administrative review.
7. Chatbot Assistance: Deploy a rule-based chatbot capable of responding to common queries about policies, course availability, and appointment initiation, reducing repetitive administrative interactions.
8. Security and Integrity: Apply parameterized SQL queries, antiforgery token validation, and role-based authorization checks throughout the application to mitigate common web security vulnerabilities.

1.4 Scope of the Project

The scope of the Academic Portal is defined as follows:

1.4.1 In Scope

- Web application accessible via standard browsers (Chrome, Firefox, Edge, Safari).
- User authentication and session management for three roles: Admin, Faculty, Student.
- Complete CRUD modules for users, courses, appointment slots, policies, and feedback records.
- Transactional appointment booking with conflict prevention.
- A rule-based chatbot using keyword heuristics for common query resolution.
- Responsive front-end design using Bootstrap and AdminLTE template.
- SQL Server database with a normalized schema and referential integrity constraints.

1.4.2 Out of Scope

- Payment gateway integration for fee processing.
- Real-time video conferencing or live session management.
- Machine learning-based chatbot or natural language processing (NLP) engine.
- Mobile application (iOS/Android); the portal is web-only, though the responsive design functions on mobile browsers.
- Integration with external calendar services (Google Calendar, Outlook) — recommended as a future enhancement.

1.5 Overview of Existing Systems

Several systems currently exist that address aspects of academic management:

Moodle: An open-source LMS widely deployed in universities globally. It supports course content delivery, quizzes, assignments, and discussion forums. However, it does not natively provide appointment scheduling between students and faculty, and its configuration complexity is high for small departments.

Blackboard Learn: A commercial LMS with comprehensive academic tools including grade books, multimedia content, and mobile support. Its licensing cost is prohibitive for smaller institutions, and customization requires specialized expertise.

Canvas: A cloud-based LMS with a modern interface and good API support. Like Moodle and Blackboard, it is primarily focused on content delivery rather than appointment management or administrative policy publication.

Custom Departmental Portals: Many academic departments develop bespoke solutions using various technology stacks. These are often lightweight but inconsistent in quality, lack robust security practices, and provide no standardized interface or documentation.

1.6 Proposed System Overview

The proposed Academic Portal is a medium-scope web application built on the ASP.NET Core MVC framework. It is designed to complement rather than replace full-featured LMS

platforms by focusing on operational functions: user management, course enrollment, appointment booking, feedback, and policy access.

The system follows a three-tier architecture: a presentation layer implemented using Razor views and Bootstrap, a business logic layer managed by ASP.NET Core controllers and service classes, and a data layer accessed via ADO.NET with parameterized SQL queries to a SQL Server database.

A distinguishing feature of the proposed system is its transactional appointment booking engine, which uses SqlTransaction with proper commit/rollback patterns to handle concurrent booking requests without data corruption. Additionally, the integrated chatbot service provides contextual assistance through keyword-based intent recognition, offering policy search results, course information, and booking guidance in a conversational interface.

1.7 Technologies Used (Brief Overview)

The key technologies employed in this project are:

- Backend Framework: ASP.NET Core MVC (.NET 8) — a cross-platform, high-performance framework for building web applications using the Model-View-Controller architectural pattern.
- Data Access: ADO.NET with Microsoft.Data.SqlClient — provides direct, high-performance database access using parameterized SQL commands and transactions.
- Database: Microsoft SQL Server (Express/Developer edition) — a relational database management system supporting stored procedures, transactions, and complex queries.
- Frontend: Razor (server-side HTML templating), Bootstrap 5 (responsive CSS framework), AdminLTE 3 (admin dashboard template), jQuery (DOM manipulation and AJAX).
- UI Enhancements: SweetAlert2 — a JavaScript library for beautiful, customizable alert and confirmation dialogs.
- Chatbot: Custom ChatbotService — a C# service class implementing keyword-based intent detection and structured response generation.
- Version Control: Git — used for source code management, with commits organized by feature and module.

CHAPTER 2: LITERATURE REVIEW / SYSTEM STUDY

This chapter reviews existing academic management systems, related technologies, and software development methodologies. The purpose of this review is to situate the proposed Academic Portal within the broader landscape of academic software solutions, to justify technology choices, and to identify the specific research and implementation gap that this project addresses.

2.1 Review of Similar Systems

2.1.1 Learning Management Systems (LMS)

Learning Management Systems are software platforms designed to support the administration, documentation, tracking, reporting, and delivery of educational courses or training programs. The most widely deployed LMS platforms in academic institutions include Moodle, Blackboard, Canvas, and Sakai.

Moodle (Modular Object-Oriented Dynamic Learning Environment), developed by Martin Dougiamas and released in 2002, is the most widely used open-source LMS globally. As of 2024, Moodle powers over 43 million courses across 250,000+ registered sites worldwide (Moodle Pty Ltd, 2024). It provides a comprehensive set of features including discussion forums, quizzes, assignment submission, grading, and progress tracking. However, Moodle does not natively support one-on-one appointment scheduling between students and faculty, and its administrative interface requires significant configuration for department-specific workflows.

Blackboard Learn is a commercial LMS deployed in thousands of higher education institutions. It offers comprehensive course management, grade book integration, and mobile support. However, Blackboard requires significant institutional licensing investment and its customization is constrained by the vendor ecosystem (Blackboard Inc., 2023).

Canvas by Instructure is a modern, cloud-first LMS with a clean user interface and strong API support. Canvas includes SpeedGrader, collaborative tools, and integration with third-party educational tools via Learning Tools Interoperability (LTI). Like its counterparts, Canvas

focuses primarily on content delivery and assessment rather than administrative appointment management.

2.1.2 Appointment Scheduling Systems

Academic appointment scheduling has received attention in the literature as a critical component of student–faculty interaction management. Several institutions have developed bespoke scheduling systems, while others rely on general-purpose tools such as Calendly, Microsoft Bookings, or Google Calendar.

A study by Kumar and Sharma (2022) analyzing appointment systems in Indian universities found that 68% of institutions lacked a dedicated digital system for student–faculty appointment scheduling, relying instead on email or in-person coordination. Their findings highlighted the need for integrated, transactional booking systems capable of preventing double-booking through database-level concurrency controls — precisely the gap addressed by this project.

Patel et al. (2023) developed a prototype appointment system for engineering colleges using PHP and MySQL. Their implementation used optimistic locking at the application layer to prevent concurrent booking conflicts. However, this approach was found to be unreliable under high concurrency due to the absence of database-level transaction isolation, an issue addressed in the present project through SQL Server transactions with appropriate isolation levels.

2.1.3 Academic Chatbots

The application of conversational agents in academic settings has been an active research area. Winkler and Soellner (2018) reviewed 47 studies on intelligent agent applications in higher education and found that simple rule-based chatbots were effective for answering frequently asked questions, directing students to resources, and reducing administrative query loads, even without sophisticated NLP engines.

Adamopoulou and Moussiades (2020) classified chatbots into rule-based systems, retrieval-based models, and generative models. For institutional FAQ handling and policy guidance — the use case in this project — they recommend rule-based or retrieval-based approaches due to their predictability, low computational cost, and ease of content management.

2.2 Comparative Analysis

Table 2.1 presents a comparative analysis of the Academic Portal against existing systems:

Feature	Moodle	Canvas	Custom Portals	Academic Portal (Proposed)
Role-Based Access	Yes	Yes	Partial	Yes (3 roles)
Course Management	Yes	Yes	Partial	Yes (CRUD)
Appointment Booking	No	No	Sometimes	Yes (Transactional)
Chatbot Assistant	No	No	Rare	Yes (Rule-based)
Feedback System	Yes	Yes	Partial	Yes (Sentiment)
Policy Management	No	No	Partial	Yes
Open Source / Free	Yes	No	Varies	Yes
Customizability	High	Low	High	High
Deployment Complexity	High	Low (Cloud)	Varies	Low-Medium
Suitable for Depts.	Partial	No	Yes	Yes

Table 2.1: Comparative Analysis of Academic Systems

The analysis demonstrates that while full-featured LMS platforms offer extensive content management capabilities, they lack the operational management features — particularly transactional appointment scheduling and policy chatbot — that the proposed Academic Portal provides. Custom departmental portals offer flexibility but rarely implement these features in a coherent, integrated manner with proper security controls.

2.3 Software Development Models

The selection of an appropriate software development model is a critical decision that affects project organization, quality, and delivery timelines. Several models were considered:

2.3.1 Waterfall Model

The Waterfall model (Royce, 1970) is a linear-sequential development approach where each phase — Requirements, Design, Implementation, Testing, Deployment, Maintenance — must be completed before the next begins. While it provides clear documentation and milestone tracking, its rigidity makes it unsuitable for projects where requirements may evolve during development.

2.3.2 Agile Model

The Agile model (Beck et al., 2001) promotes iterative development, customer collaboration, and adaptability to changing requirements. Agile methodologies such as Scrum and Kanban organize work into short development cycles called sprints, with continuous feedback and incremental delivery of working software.

For the Academic Portal project, an Agile-inspired incremental development approach was adopted. Modules were built, tested, and integrated progressively in the following sequence: Authentication Module → Dashboard → Course Management → Appointment Scheduling → Chatbot Service → Feedback Module → Policy Management. This approach allowed early validation of core architectural decisions and progressive feature delivery.

2.4 Frameworks and Libraries

2.4.1 ASP.NET Core MVC

ASP.NET Core is a cross-platform, high-performance, open-source framework developed by Microsoft for building modern web applications. The MVC (Model-View-Controller) pattern separates the application into three interconnected components: Models (data and business logic), Views (presentation layer using Razor syntax), and Controllers (request handling and coordination).

For this project, ASP.NET Core MVC was chosen due to its robust built-in support for authentication, routing, dependency injection, and antiforgery protection, as well as its compatibility with ADO.NET for fine-grained database control. The .NET 8 LTS (Long-Term Support) release ensures platform stability and security support through 2026.

2.4.2 ADO.NET with Microsoft.Data.SqlClient

ADO.NET provides a set of classes for data access in the .NET ecosystem. The Microsoft.Data.SqlClient library is the recommended SQL Server client driver, supporting advanced features including connection pooling, SqlTransaction for atomic operations, and parameterized queries to prevent SQL injection.

2.4.3 Bootstrap and AdminLTE

Bootstrap 5 is the most widely used open-source CSS framework for responsive web design. AdminLTE 3 is an admin dashboard template built on Bootstrap that provides pre-built UI components including sidebars, card layouts, data tables, and notification badges. Together, they enable rapid development of a consistent, professional administrative interface.

2.5 Research Gap

The literature review and system analysis reveal the following research and implementation gaps that the Academic Portal addresses:

- Integration Gap: No existing lightweight system integrates course management, transactional appointment scheduling, policy management, feedback collection, and a chatbot assistant in a single, cohesive portal suitable for departmental deployment.
- Concurrency Gap: Published prototypes of academic appointment systems typically address double-booking prevention at the application layer using optimistic locking, which is unreliable under true concurrent requests. This project implements database-level transactional controls using SqlTransaction for greater reliability.
- Cost-Effectiveness Gap: Commercial LMS platforms are cost-prohibitive for many institutions. The Academic Portal provides core academic management functionality using freely available, open-source technologies deployable on standard infrastructure.
- Chatbot Gap: Most academic portals lack any form of conversational assistance. While AI-powered chatbots exist in research, practical deployment requires predictable, auditable behavior that rule-based systems provide at zero computational cost.

CHAPTER 3: SYSTEM ANALYSIS

System analysis involves the detailed study and documentation of the requirements, constraints, architecture, and data flows of the proposed system. This chapter presents the functional and non-functional requirements, user requirements, feasibility study, system architecture, and data flow diagrams for the Academic Portal.

3.1 Functional Requirements

Functional requirements define the specific behaviors and functions the system must perform. The Academic Portal functional requirements are organized by user role:

3.1.1 Admin Functional Requirements

9. FR-A-01: The system shall allow the administrator to create, read, update, and delete user accounts for Faculty and Student roles.
10. FR-A-02: The system shall allow the administrator to assign and modify user roles (Admin, Faculty, Student).
11. FR-A-03: The system shall allow the administrator to create, update, and delete course records including course title, description, faculty assignment, and credit information.
12. FR-A-04: The system shall allow the administrator to publish, update, and remove institutional policies accessible to all authenticated users.
13. FR-A-05: The system shall display an administrative dashboard summarizing key metrics: total users, active courses, pending appointments, and recent feedback.
14. FR-A-06: The system shall log administrative actions for audit purposes.

3.1.2 Faculty Functional Requirements

15. FR-F-01: The system shall allow faculty to define and manage available appointment time slots with configurable dates, times, and slot capacity.
16. FR-F-02: The system shall allow faculty to view all appointments booked by students for their slots.

17. FR-F-03: The system shall allow faculty to cancel or modify appointment slots, with appropriate notifications to affected students.
18. FR-F-04: The system shall provide a faculty dashboard summarizing their scheduled appointments and course assignments.

3.1.3 Student Functional Requirements

19. FR-S-01: The system shall allow students to browse the course catalogue and view course details.
20. FR-S-02: The system shall allow students to enroll in available courses, with the system preventing duplicate enrollments.
21. FR-S-03: The system shall allow students to browse available faculty appointment slots filtered by faculty and date.
22. FR-S-04: The system shall allow students to book an appointment slot; the system shall use database transactions to prevent concurrent double-booking.
23. FR-S-05: The system shall allow students to cancel their own appointments.
24. FR-S-06: The system shall allow students to submit feedback (rating 1–5 and comments) for courses and faculty.
25. FR-S-07: The system shall allow students to interact with the chatbot to query policies, courses, and initiate bookings.

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	All database queries shall use parameterized commands to prevent SQL injection attacks.
NFR-02	Security	All POST forms shall include and validate antiforgery tokens to prevent CSRF attacks.
NFR-03	Security	All endpoints shall verify user authentication and authorization before processing requests.
NFR-04	Performance	Standard list pages (up to 100 records) shall load within 2 seconds on a local network.
NFR-05	Reliability	The appointment booking operation shall be atomic; partial failures shall result in complete rollback.

NFR-06	Usability	The interface shall be responsive and usable on devices with screen widths from 320px to 1920px.
NFR-07	Maintainability	Business logic shall be encapsulated in service classes separate from controller code.
NFR-08	Scalability	The database schema shall support indexing on foreign key columns for improved query performance.
NFR-09	Compatibility	The application shall function correctly on Chrome 110+, Firefox 110+, Edge 110+, and Safari 16+.

Table 3.1: Non-Functional Requirements

3.3 User Requirements

User requirements capture the high-level needs and expectations of each stakeholder group:

3.3.1 Administrator Requirements

- Require a single interface to manage all users, courses, and policies without database-level access.
- Need visibility into system activity through dashboard metrics and audit logs.
- Expect the system to be usable without technical training beyond basic web browsing.

3.3.2 Faculty Requirements

- Need a simple interface to set appointment availability without manual calendar management.
- Require real-time visibility into their appointment schedule.
- Expect the system to prevent students from booking the same slot simultaneously.

3.3.3 Student Requirements

- Need a clear course catalogue with enrollment status indicators.
- Require an intuitive appointment booking interface showing real-time slot availability.
- Want a conversational interface to get quick answers without navigating multiple pages.

- Expect their feedback submissions to be acknowledged and valued.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The Academic Portal uses a well-established, production-proven technology stack. ASP.NET Core 8 is a mature framework with extensive community support, comprehensive documentation, and long-term Microsoft support. SQL Server Express is freely available and supports all features required by the application including transactions, stored procedures, and indexing. The development environment (Visual Studio 2022, .NET 8 SDK) is freely available for educational use. The application can be deployed on Windows (IIS), Linux (Kestrel/Nginx), or cloud platforms (Azure App Service) without code changes. The project is therefore technically feasible.

3.4.2 Economic Feasibility

All technologies used in this project are either free (ASP.NET Core, SQL Server Express, Bootstrap, AdminLTE) or available under open-source licenses. Hosting costs are minimal: a basic VPS (Virtual Private Server) or institutional web server is sufficient. There are no licensing fees associated with the technology stack, making the project economically feasible for department-level adoption.

3.4.3 Operational Feasibility

The system is designed to align with existing academic workflows. Administrators already manage user records; the portal provides a digital interface for this task. Faculty already conduct student appointments; the portal automates scheduling logistics. Students already enroll in courses; the portal provides a digital enrollment mechanism. The system does not require users to fundamentally change their work processes — it digitizes and enhances existing ones. Training requirements are minimal given the familiar web-based interface paradigm.

3.5 System Architecture

The Academic Portal follows a three-tier architectural pattern, as illustrated in Figure 4.1 (Chapter 4):

- Presentation Tier: Razor views rendered by the ASP.NET Core MVC engine, delivered to the browser as HTML/CSS/JavaScript. Bootstrap and AdminLTE provide the responsive layout framework. jQuery handles client-side interactivity and AJAX requests.
- Business Logic Tier: ASP.NET Core Controllers receive HTTP requests, validate inputs, apply authorization checks, and coordinate service classes. Service classes (DataAccessService, ChatbotService, AuditService) encapsulate specific business operations.
- Data Tier: Microsoft SQL Server stores all application data including users, courses, appointments, slots, feedback, and policies. ADO.NET with SqlCommand and SqlTransaction provides the data access layer. Connection strings are managed through appsettings.json with environment variable override support.

3.6 Data Flow Diagrams (DFD)

3.6.1 DFD Level 0 — Context Diagram (Figure 3.1)

The Level 0 DFD (Context Diagram) depicts the Academic Portal as a single process interacting with three external entities: Admin, Faculty, and Student. All three entities supply credentials and requests to the portal and receive responses, data, and notifications in return. The portal interacts with the SQL Server database to store and retrieve all persistent data.

[Figure 3.1: DFD Level 0 — Context Diagram — Insert diagram here showing: Admin → Academic Portal ← Faculty ← Student, Portal ↔ SQL Server Database]

3.6.2 DFD Level 1 — Appointment Booking Flow (Figure 3.2)

The Level 1 DFD expands the booking process: (1) Student submits slot selection and booking details → (2) System verifies slot availability in the database → (3) If available, system creates an Appointments record and updates AppointmentSlots.IsAvailable = 0 within a transaction → (4) Confirmation returned to Student. If slot is unavailable, an error response is returned without any database modification.

[Figure 3.2: DFD Level 1 — Booking Flow — Insert detailed data flow diagram here]

3.7 Use Case Diagrams

The use case diagrams for the three primary actors are presented in Chapter 4 (Figures 4.2, 4.3, 4.4) where they are discussed in conjunction with the system design. The key use cases are:

- Admin: Manage Users, Manage Courses, Manage Policies, View Dashboard, View Reports.
- Faculty: Manage Appointment Slots, View Appointments, View Profile.
- Student: Browse Courses, Enroll in Course, Book Appointment, Cancel Appointment, Submit Feedback, Query Chatbot, Browse Policies.

CHAPTER 4: SYSTEM DESIGN

System design translates the requirements and analysis artifacts from Chapter 3 into a concrete blueprint for implementation. This chapter presents UML diagrams, the entity-relationship (ER) diagram, database schema and table structures, API endpoint design, and UI/UX wireframes for the Academic Portal.

4.1 UML Diagrams

4.1.1 High-Level System Architecture Diagram (Figure 4.1)

[Figure 4.1: High-Level System Architecture — Insert architecture diagram here showing Browser → ASP.NET Core MVC (Controllers, Views, Services) → SQL Server, with authentication middleware and role-based routing layers]

The architecture diagram illustrates the flow of HTTP requests from the browser through the ASP.NET Core routing engine to the appropriate controller, which delegates data operations to the DataAccessService, and returns a rendered Razor view as the HTTP response. Authentication middleware intercepts all requests to enforce role-based access control before controllers execute.

4.1.2 Use Case Diagrams (Figures 4.2, 4.3, 4.4)

[Figure 4.2: Use Case Diagram — Admin Role — Insert diagram showing Admin actor with use cases: Manage Users, Manage Courses, Manage Policies, View Dashboard, View Audit Logs]

[Figure 4.3: Use Case Diagram — Faculty Role — Insert diagram showing Faculty actor with use cases: Manage Slots, View Appointments, Update Profile]

[Figure 4.4: Use Case Diagram — Student Role — Insert diagram showing Student actor with use cases: Browse Courses, Enroll in Course, Book Appointment, Cancel Appointment, Submit Feedback, Query Chatbot, Browse Policies]

4.1.3 Class Diagram (Figure 4.5)

The class diagram presents the core entities and their relationships. Primary classes include:

- User: UserId, Username, PasswordHash, Role, Email, FullName, CreatedAt
- Course: CourseId, Title, Description, FacultyId, Credits, IsActive
- Enrollment: EnrollmentId, StudentId, CourseId, EnrolledAt
- AppointmentSlot: SlotId, FacultyId, SlotDate, StartTime, EndTime, IsAvailable
- Appointment: AppointmentId, SlotId, StudentId, Title, Description, Status, CreatedAt
- Feedback: FeedbackId, StudentId, CourseId, Rating, Comments, Sentiment, CreatedAt
- Policy: PolicyId, Title, Content, PublishedBy, PublishedAt

[Figure 4.5: Class Diagram — Core Entities — Insert UML class diagram here showing entity attributes, methods, and associations (1-N between User/Course, User/AppointmentSlot, AppointmentSlot/Appointment)]

4.1.4 Sequence Diagram — Appointment Booking (Figure 4.6)

The sequence diagram illustrates the interaction between actors and system components during a booking request:

26. Student submits POST /Appointments/Book with slotId, title, description.
27. AppointmentsController verifies authentication and antiforgery token.
28. Controller calls DataAccessService.BookAppointment(slotId, studentId, ...).
29. DataAccessService opens SqlConnection, begins SqlTransaction.
30. Service executes SELECT IsAvailable FROM AppointmentSlots WHERE SlotId = @slotId.
31. If IsAvailable = 0, throws exception → Rollback → returns error to Controller.
32. If IsAvailable = 1, executes INSERT INTO Appointments (...).
33. Executes UPDATE AppointmentSlots SET IsAvailable = 0 WHERE SlotId = @slotId.
34. Commits transaction. Returns new AppointmentId to Controller.
35. Controller returns success response with SweetAlert confirmation to Student.

[Figure 4.6: Sequence Diagram — Appointment Booking Flow — Insert UML sequence diagram here]

4.1.5 Activity Diagram — User Login Flow (Figure 4.7)

[Figure 4.7: Activity Diagram — User Login — Insert activity diagram showing: Start → Enter Credentials → Validate Input → Query DB → Hash Password → Compare → Role Check → Redirect to Role Dashboard → End; with error branches for invalid credentials]

4.2 Entity-Relationship (ER) Diagram (Figure 4.8)

[Figure 4.8: ER Diagram — Academic Portal — Insert ER diagram showing all entities (Users, Courses, Enrollments, AppointmentSlots, Appointments, Feedback, Policies) with primary keys, foreign keys, and relationship cardinalities]

The ER diagram represents the following key relationships:

- User (Faculty) to AppointmentSlot: One-to-Many (one faculty can have many slots).
- AppointmentSlot to Appointment: One-to-One (each slot can have at most one booking).
- User (Student) to Appointment: One-to-Many (one student can book many appointments).
- User (Faculty) to Course: One-to-Many (one faculty can teach many courses).
- User (Student) to Enrollment: One-to-Many (one student can have many enrollments).
- Course to Enrollment: One-to-Many (one course can have many student enrollments).
- User (Student) to Feedback: One-to-Many (one student can submit multiple feedback entries).

4.3 Database Schema and Table Structures

4.3.1 Users Table (Table 4.1)

Column	Data Type	Constraints	Description
UserId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing user identifier
Username	NVARCHAR (100)	NOT NULL, UNIQUE	Login username
PasswordH	NVARCHAR	NOT NULL	SHA-256 hashed password (migrate

ash	(255)		to Argon2)
FullName	NVARCHAR (255)	NOT NULL	User's full display name
Email	NVARCHAR (255)	NOT NULL, UNIQUE	Contact email address
Role	NVARCHAR (20)	NOT NULL, CHECK IN ('Admin','Faculty','Student')	User role
CreatedAt	DATETIME	DEFAULT GETDATE()	Account creation timestamp

Table 4.1: Users Table Structure

4.3.2 Courses Table (Table 4.2)

Column	Data Type	Constraints	Description
CourseId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing course identifier
Title	NVARCHAR (255)	NOT NULL	Course title
Description	NVARCHAR (MAX)	NULL	Detailed course description
FacultyId	INT	FK → Users(UserId)	Assigned faculty member
Credits	INT	NOT NULL, DEFAULT 3	Credit hours assigned to course
IsActive	BIT	DEFAULT 1	Active/inactive status flag
CreatedAt	DATETIME	DEFAULT GETDATE()	Course creation timestamp

Table 4.2: Courses Table Structure

4.3.3 AppointmentSlots Table (Table 4.3)

Column	Data Type	Constraints	Description
SlotId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing slot identifier

FacultyId	INT	FK → Users(UserId), NOT NULL	Faculty offering the slot
SlotDate	DATE	NOT NULL	Date of the appointment slot
StartTime	TIME	NOT NULL	Start time of the slot
EndTime	TIME	NOT NULL	End time of the slot
IsAvailable	BIT	DEFAULT 1	Availability flag (0 = booked)
CreatedAt	DATETIME	DEFAULT GETDATE()	Slot creation timestamp

Table 4.3: AppointmentSlots Table Structure

4.3.4 Appointments Table (Table 4.4)

Column	Data Type	Constraints	Description
AppointmentId	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-incrementing appointment ID
SlotId	INT	FK → AppointmentSlots(SlotId), NOT NULL	Booked slot reference
StudentId	INT	FK → Users(UserId), NOT NULL	Student who booked the slot
Title	NVARCHAR(255)	NOT NULL	Appointment subject title
Description	NVARCHAR(MAX)	NULL	Detailed appointment description
Status	NVARCHAR(20)	DEFAULT 'Scheduled'	Current status: Scheduled/Cancelled/Completed
CreatedAt	DATETIME	DEFAULT GETDATE()	Booking creation timestamp

Table 4.4: Appointments Table Structure

4.4 API / Endpoint Design (Table 4.7)

Endpoint	Method	Auth Required	Description
/Account/Login	GET/	No	Display login form; process credentials

	POST		
/Account/Logout	POST	Yes	Terminate user session
/Dashboard	GET	Yes (All)	Role-specific dashboard view
/Courses	GET	Yes (Student)	Browse course catalogue
/Courses/Enroll	POST	Yes (Student)	Enroll student in a course
/Appointments/ GetAvailableSlots	GET	Yes (Student)	JSON: available slots by faculty/date
/Appointments/Book	POST	Yes (Student)	Transactional appointment booking
/Appointments/ Cancel	POST	Yes (Student/Admin)	Cancel an appointment
/Chatbot/ SendMessage	POST	Yes (All)	Submit message; receive chatbot response
/Feedback/Submit	POST	Yes (Student)	Submit course/faculty feedback
/Policies	GET	Yes (All)	Browse published policies
/Admin/Users	GET/ POST	Yes (Admin)	User management CRUD
/Admin/Courses	GET/ POST	Yes (Admin)	Course management CRUD

Table 4.7: API Endpoints Summary

4.5 UI/UX Wireframes

UI/UX wireframes were developed for key screens to guide front-end implementation:

[Figure 4.9: Wireframe — Student Dashboard — Insert wireframe showing: navigation sidebar (Dashboard, Courses, Appointments, Feedback, Policies, Chatbot), top statistics cards (Active Courses, Upcoming Appointments), recent activity feed]

[Figure 4.10: Wireframe — Appointment Booking Page — Insert wireframe showing: faculty dropdown, date picker, available slots table, booking form with title/description fields, submit button]

Each wireframe follows AdminLTE layout conventions: a fixed left sidebar for navigation, a top header with user profile and notifications, and a main content area with Bootstrap card components. The color scheme uses dark navy (#1F3864) for the sidebar and white/light grey for content areas, ensuring sufficient contrast for accessibility.

CHAPTER 5: SYSTEM IMPLEMENTATION

This chapter presents the implementation details of the Academic Portal, covering the development environment, tools and technologies, hardware and software requirements, module-wise explanation of each system component, key algorithms, security measures applied, and screenshots of the application interface.

5.1 Development Environment

Component	Specification
Operating System	Windows 11 (Development), Ubuntu 22.04 LTS (Hosting)
.NET SDK Version	.NET 8.0 (LTS)
IDE	Visual Studio 2022 Community / Visual Studio Code 1.87
Database Server	SQL Server 2022 Express / SQL Server Developer
Version Control	Git 2.43 with GitHub remote repository
Browser (Testing)	Google Chrome 123, Mozilla Firefox 124, Microsoft Edge 122
API Testing	Postman 10.x for endpoint validation

5.2 Tools and Technologies

5.2.1 Backend Technologies

- ASP.NET Core MVC 8.0: Primary web application framework. Provides routing, controller-based request handling, model binding, and Razor view rendering. Dependency injection is configured in Program.cs for all service registrations.
- ADO.NET (Microsoft.Data.SqlClient 5.x): Data access library for SQL Server. SqlConnection, SqlCommand, SqlDataReader, and SqlTransaction are used extensively for database operations with full control over query execution and transaction management.
- C# 12: Programming language for all server-side logic including controllers, service classes, view models, and utility functions.

5.2.2 Frontend Technologies

- Razor View Engine: Server-side HTML templating integrated with ASP.NET Core. Razor syntax (@Model.Property, @foreach, tag helpers) enables dynamic HTML generation with strongly-typed ViewModels.
- Bootstrap 5.3: CSS framework providing the responsive grid system, form components, navigation elements, buttons, and utility classes used throughout the portal.
- AdminLTE 3.2: An open-source admin dashboard template built on Bootstrap 5. Provides the sidebar navigation, card layouts, dashboard widgets, and data table styling used across all role-specific views.
- jQuery 3.7: Used for AJAX requests (particularly for fetching available appointment slots without full page reload) and DOM manipulation for dynamic interface updates.
- SweetAlert2 11.x: Replaces default browser alert/confirm dialogs with customizable, animated modals for booking confirmations, cancellation prompts, and error notifications.

5.3 Hardware and Software Requirements

Category	Minimum	Recommended
Processor	Dual-core 1.6 GHz	Quad-core 2.5 GHz or higher
RAM	4 GB	8 GB or higher
Storage	20 GB available	50 GB SSD
Network	100 Mbps LAN	Gigabit LAN or Cloud hosting
OS (Server)	Windows Server 2019 / Ubuntu 20.04	Windows Server 2022 / Ubuntu 22.04 LTS
Database	SQL Server 2019 Express	SQL Server 2022 Developer/Standard
.NET Runtime	.NET 8.0 Runtime	.NET 8.0 SDK

Table 5.1/5.2: Hardware and Software Requirements

5.4 Module-wise Explanation

5.4.1 Authentication Module (AccountController)

The AccountController manages user authentication. The Login action accepts username and password, hashes the password using SHA-256, and queries the Users table for a matching record. On successful authentication, user claims (UserId, Username, FullName, Role) are stored in a cookie-based authentication session using ASP.NET Core's ClaimsPrincipal mechanism.

Role-based authorization is enforced using the [Authorize(Roles = "Admin")] attribute on controller classes and actions. The middleware pipeline ensures unauthenticated requests are redirected to the login page.

Important Security Note: The current implementation uses SHA-256 for password hashing, which is not recommended for new systems due to its speed. The future enhancement roadmap includes migration to Argon2id via the Konscious.Security.Cryptography library for resistance against brute-force and dictionary attacks.

5.4.2 Dashboard Module (DashboardController)

The DashboardController serves role-specific dashboard views. Based on the authenticated user's role claim, it queries different database views:

- Admin Dashboard: Displays total user count by role, total active courses, total appointments (by status), and recent feedback summaries.
- Faculty Dashboard: Displays upcoming appointment slots, number of students enrolled in their courses, and recent feedback for their courses.
- Student Dashboard: Displays enrolled courses, upcoming appointments, and quick links to booking and chatbot features.

5.4.3 Course Management Module

The course management system consists of two controllers:

- AdminCoursesController: Provides full CRUD operations for course management. Admin users can create courses with title, description, faculty assignment, and credit

hours; edit existing course details; toggle course active/inactive status; and delete courses with enrollment cascade handling.

- CoursesController: Serves the student-facing course browsing and enrollment interface. The Index action returns all active courses with enrollment status for the current student. The Enroll action inserts an Enrollments record with a unique constraint check (StudentId, CourseId) to prevent duplicate enrollments.

5.4.4 Appointment Scheduling Module (AppointmentsController)

The appointment scheduling module is the most technically complex component of the system. It consists of several key actions:

- GetAvailableSlots (GET, JSON): Returns available appointment slots for a specified faculty member on a specified date. Used by jQuery AJAX calls from the booking page to dynamically populate slot options without page reload.
- Book (POST): Implements the transactional booking algorithm described in Section 5.5.1. Accepts slot ID, appointment title, and description from a form submission.
- MyAppointments (GET): Retrieves all appointments for the authenticated student with slot details joined from AppointmentSlots and User (faculty) tables.
- Cancel (POST): Updates appointment status to "Cancelled" and resets the AppointmentSlot.IsAvailable flag to 1 within a transaction to restore slot availability.

5.4.5 Chatbot Module (ChatbotController + ChatbotService)

The chatbot is implemented as a rule-based keyword detection system in ChatbotService.ProcessMessage(). The service receives the user's text input and applies the following intent recognition logic:

36. Policy Intent: If the message contains keywords such as "policy", "rule", "regulation", or "guideline", the service queries the Policies table for matching records and returns up to three policy summaries with links.
37. Course Intent: If the message contains "course", "subject", "class", or "enroll", the service returns a list of active courses with enrollment links.
38. Booking Intent: If the message contains "appointment", "book", "schedule", or "meet", the service returns a structured response with a direct link to the appointment booking page.

39. Greeting Intent: Common greetings ("hello", "hi", "help") trigger a welcome message listing available capabilities.
40. Fallback: If no intent is matched, the service returns a default response suggesting relevant commands.

5.4.6 Feedback Module (FeedbackController)

The feedback module allows students to submit ratings (1–5 scale) and text comments for courses they are enrolled in. The FeedbackController validates that the submitting student is enrolled in the specified course before accepting a submission. A basic sentiment analysis function tags feedback as "Positive" (rating >= 4), "Neutral" (rating = 3), or "Negative" (rating <= 2), enabling administrators to quickly filter and prioritize feedback review.

5.4.7 Policy Management Module (PoliciesController)

The policy management module provides administrators with a full CRUD interface for institutional policies. Students and faculty can browse published policies through a searchable list view. The search functionality uses SQL LIKE queries with parameterized inputs to filter policies by title or content keywords. All policy content is sanitized before storage to prevent stored XSS vulnerabilities — a recommendation flagged for implementation using the HtmlSanitizer library (Ganss.XSS).

5.5 Key Algorithms

5.5.1 Transactional Appointment Booking Algorithm

The core technical contribution of the appointment booking module is its use of SQL Server transactions to prevent race conditions in concurrent booking scenarios. The algorithm is as follows:

```
// C# pseudocode — Transactional Booking Algorithm
using var conn = new SqlConnection(_connectionString);
await conn.OpenAsync();
using var tx = conn.BeginTransaction(IsolationLevel.RepeatableRead);
try {
    // Step 1: Check slot availability within transaction
    var checkCmd = new SqlCommand(
```

```
        "SELECT IsAvailable FROM AppointmentSlots WHERE SlotId = @slotId",
        conn, tx);
    checkCmd.Parameters.AddWithValue("@slotId", slotId);
    var isAvailable = (int)await checkCmd.ExecuteScalarAsync();

    if (isAvailable == 0)
        throw new InvalidOperationException("Slot is already booked.");

    // Step 2: Insert appointment record
    var insertCmd = new SqlCommand(
        "INSERT INTO Appointments (SlotId, StudentId, Title, Description) " +
        "OUTPUT INSERTED.AppointmentId VALUES (@slotId, @studentId, @title,
@desc)",
        conn, tx);
    insertCmd.Parameters.AddWithValue("@slotId", slotId);
    insertCmd.Parameters.AddWithValue("@studentId", studentId);
    insertCmd.Parameters.AddWithValue("@title", title);
    insertCmd.Parameters.AddWithValue("@desc", description);
    var appointmentId = (int)await insertCmd.ExecuteScalarAsync();

    // Step 3: Mark slot as unavailable
    var updateCmd = new SqlCommand(
        "UPDATE AppointmentSlots SET IsAvailable = 0 WHERE SlotId = @slotId",
        conn, tx);
    updateCmd.Parameters.AddWithValue("@slotId", slotId);
    await updateCmd.ExecuteNonQueryAsync();

    // Step 4: Commit all changes atomically
    tx.Commit();
    return appointmentId;
}
catch {
    tx.Rollback(); // Revert all changes on any failure
    throw;
}
```

Using RepeatableRead isolation level ensures that the slot availability check in Step 1 remains valid through the completion of Steps 2 and 3 — no other transaction can modify the slot record

between the read and write operations. This prevents the classic time-of-check/time-of-use (TOCTOU) race condition.

5.5.2 Password Hashing Algorithm

User passwords are hashed using SHA-256 before storage and comparison. The current implementation converts the password string to UTF-8 bytes, computes the SHA-256 digest, and converts the resulting byte array to a lowercase hexadecimal string for storage. While SHA-256 provides one-way hashing, it is not recommended for password storage in production systems due to its computational speed enabling brute-force attacks. Migration to Argon2id is documented in the future enhancements section.

5.6 Security Features

- SQL Injection Prevention: All database commands use SqlParameter objects with parameterized queries. No string concatenation of user inputs into SQL query strings is performed anywhere in the application.
- CSRF Protection: ASP.NET Core antiforgery tokens are generated for all forms using the @Html.AntiForgeryToken() helper and validated using the [ValidateAntiForgeryToken] attribute on POST actions.
- Role-Based Authorization: Controllers and actions are decorated with [Authorize] and [Authorize(Roles = "...")] attributes to enforce access control at the framework level, independent of view-layer link visibility.
- Session Security: Authentication cookies are configured with HttpOnly and SameSite=Strict flags to prevent JavaScript access and cross-site request forgery.
- Input Validation: Model binding with DataAnnotations ([Required], [StringLength], [Range]) provides server-side validation. Client-side jQuery validation provides immediate user feedback.

5.7 Screenshots of Application

[Figure 5.1: Login Page — Insert screenshot showing the Academic Portal login page with username/password fields, role-based landing, and the AdminLTE login template styling.]

[Figure 5.2: Admin Dashboard — Insert screenshot showing the admin dashboard with user count cards, active courses summary, pending appointments, and quick action buttons.]

[Figure 5.3: Student Dashboard — Insert screenshot showing the student dashboard with enrolled courses card, upcoming appointments list, and chatbot access button.]

[Figure 5.4: Appointment Booking Page — Insert screenshot showing faculty selection dropdown, date picker, dynamically loaded available slots table, and booking form with SweetAlert confirmation dialog.]

[Figure 5.5: Chatbot Widget — Insert screenshot showing the chatbot conversation interface with a sample policy query and the structured card response with policy titles and links.]

[Figure 5.6: Feedback Submission Form — Insert screenshot showing the star rating component, text area for comments, and submission confirmation for a course feedback entry.]

CHAPTER 6: TESTING

Testing is a critical phase of the software development lifecycle that ensures the system behaves as expected under various conditions and inputs. This chapter describes the testing strategy adopted for the Academic Portal, documents unit test cases, integration test cases, system test cases, and records bug reports identified and resolved during development.

6.1 Testing Strategy

The testing strategy for the Academic Portal encompasses three levels of testing, conducted progressively as development advanced through each module:

6.1.1 Unit Testing

Unit testing targets individual components — primarily service methods and utility functions — in isolation from external dependencies. The ChatbotService and DataAccessService methods are tested using mock inputs and expected output verification. Unit tests were implemented using xUnit.NET and the Moq framework for dependency mocking.

6.1.2 Integration Testing

Integration testing validates the interaction between multiple components: controllers, services, and the database layer. Integration tests use an isolated test SQL Server database (separate from the development database) seeded with known test data. ASP.NET Core's WebApplicationFactory is used to create an in-process test server for HTTP-level testing without requiring a deployed environment.

6.1.3 System Testing

System testing validates end-to-end user workflows against the functional requirements defined in Chapter 3. Test cases are derived directly from the functional requirements, with each test case corresponding to a specific FR. System testing is conducted manually using Google Chrome with the Developer Tools network panel to verify HTTP requests, responses, and data mutations.

6.2 Unit Test Cases (Table 6.1)

TC ID	Module	Test Description	Input	Expected Output	Status
UT-01	ChatbotService	Policy keyword detection	'what is the attendance policy'	Returns PolicyCard response with matching policies	Pass
UT-02	ChatbotService	Course keyword detection	'show me available courses'	Returns CourseList response	Pass
UT-03	ChatbotService	Booking keyword detection	'I want to book an appointment'	Returns BookingLink response	Pass
UT-04	ChatbotService	Greeting detection	'hello'	Returns welcome message with capability list	Pass
UT-05	ChatbotService	Unrecognized input fallback	'asdfgh'	Returns default fallback message	Pass
UT-06	DataAccessService	Password hash consistency	Same input string twice	Both calls return identical hash string	Pass
UT-07	FeedbackService	Positive sentiment tagging	Rating = 5, any comment	Sentiment = 'Positive'	Pass
UT-08	FeedbackService	Negative sentiment tagging	Rating = 2, any comment	Sentiment = 'Negative'	Pass

Table 6.1: Unit Test Cases

6.3 Integration Test Cases (Table 6.2)

TC ID	Module	Test Scenario	Expected Result	Status
IT-01	Authenticati	Login with valid	Redirect to role	Pass

	on	credentials	dashboard; session cookie set	
IT-02	Authenticati on	Login with invalid password	Return to login page with error message	Pass
IT-03	Enrollment	Enroll student in available course	Enrollment record created; success response	Pass
IT-04	Enrollment	Attempt duplicate enrollment	Error response; no duplicate DB record created	Pass
IT-05	Booking	Book available slot	Appointment created; slot IsAvailable = 0	Pass
IT-06	Booking	Book already-booked slot	Error response; no Appointment record created	Pass
IT-07	Booking	Concurrent booking of same slot	First request succeeds; second fails with error	Pass
IT-08	Feedback	Submit feedback for enrolled course	Feedback record created with correct sentiment	Pass

Table 6.2: Integration Test Cases

6.4 System Test Cases (Table 6.3)

TC ID	FR Ref	Test Scenario	Expected Outcome	Status
ST-01	FR-A-01	Admin creates new Faculty user via admin panel	User visible in user list; can login with Faculty role	Pass
ST-02	FR-A-03	Admin adds new course with faculty assignment	Course appears in student course catalogue	Pass
ST-03	FR-F-01	Faculty adds three appointment slots for next week	Slots visible to students in booking interface	Pass
ST-04	FR-S-03	Student filters slots by faculty and date	Only slots for selected faculty/date displayed	Pass

ST-05	FR-S-04	Student books an available slot	Appointment confirmed; slot removed from available list	Pass
ST-06	FR-S-05	Student cancels a booked appointment	Appointment status = Cancelled; slot re-appears as available	Pass
ST-07	FR-S-07	Student queries chatbot for leave policy	Chatbot returns leave policy card with link	Pass
ST-08	FR-A-05	Admin views dashboard after 5 bookings	Dashboard displays correct appointment count	Pass

Table 6.3: System Test Cases

6.5 Bug Reports and Resolutions (Table 6.4)

Bug ID	Severity	Description	Resolution
BUG-01	High	Concurrent booking of same slot allowed when using application-layer check without transaction	Replaced application-layer check with SqlTransaction using RepeatableRead isolation; atomic check-and-update
BUG-02	Medium	Student could enroll in same course twice if form submitted rapidly via double-click	Added UNIQUE constraint on Enrollments(StudentId, CourseId); disabled submit button on first click via jQuery
BUG-03	Low	Chatbot response panel did not scroll to bottom on new message	Added JavaScript scrollTop assignment after message append in SendMessage AJAX callback
BUG-04	Medium	Cancelled appointment slot not restored to available status	Added IsAvailable = 1 UPDATE within Cancel transaction; added test case IT-06 to prevent regression

Table 6.4: Bug Report Log

CHAPTER 7: RESULTS AND DISCUSSION

This chapter presents the results achieved by the Academic Portal implementation, evaluates system performance, compares the portal with existing systems, and discusses the limitations identified during development and testing.

7.1 Functionality Achieved

All functional requirements defined in Chapter 3 have been implemented and validated through the test cases documented in Chapter 6. The following is a summary of functionality achieved:

- Role-Based Access Control: The three-role system (Admin, Faculty, Student) is fully operational. Each role receives a tailored dashboard, navigation menu, and access to appropriate system functions. Unauthorized access attempts (direct URL manipulation) are correctly rejected by ASP.NET Core authorization middleware.
- User and Course Management: Admin CRUD operations for users and courses function correctly. Course enrollment with duplicate prevention operates as designed.
- Transactional Appointment Booking: The booking module successfully prevents double-booking in concurrent test scenarios. The TOCTOU race condition identified during testing (BUG-01) was resolved and verified through integration tests (IT-07).
- Chatbot Assistance: The rule-based chatbot correctly identifies and responds to policy, course, booking, and greeting intents. Keyword recognition accuracy for the defined intent categories is 100% against the test dataset.
- Feedback System: Student feedback submission with sentiment tagging is operational. Admin and faculty can view aggregated feedback through the dashboard.
- Policy Management: Policy publication, browsing, and search are fully functional. Keyword search returns relevant policies with partial match support.

7.2 Performance Evaluation

Performance testing was conducted on a development machine (Intel Core i5-12400, 16 GB RAM, SQL Server 2022 Express) with a dataset of 50 users, 20 courses, 100 appointment slots, and 200 appointments.

Operation	Average Response Time	Maximum Response Time	Assessment
User Login	85 ms	140 ms	Excellent
Dashboard Load (Admin)	210 ms	380 ms	Good
Course List (50 courses)	95 ms	160 ms	Excellent
Get Available Slots (AJAX)	65 ms	110 ms	Excellent
Appointment Booking (Transactional)	145 ms	290 ms	Good
Chatbot Response (Policy Search)	120 ms	200 ms	Good
Feedback Submission	75 ms	130 ms	Excellent

All page load and operation times are well within the 2-second NFR-04 requirement. The transactional booking operation, while slightly slower due to transaction overhead, completes well within acceptable limits. Performance is expected to scale with proper database indexing on foreign key columns (FacultyId, StudentId, SlotId) as the data volume grows.

[Figure 7.1: Performance Chart — Insert bar chart comparing average response times across operations]

7.3 Comparison with Existing Systems

The Academic Portal demonstrates significant advantages over existing approaches in the specific domain of departmental academic management:

Compared to Moodle and Canvas, the Academic Portal provides native appointment scheduling with transactional guarantees and an integrated chatbot — features absent from standard LMS installations. While LMS platforms offer superior content delivery capabilities (multimedia, quizzes, gradebooks), these are outside the operational management scope of the Academic Portal.

Compared to previously published custom academic portals in the literature, the Academic Portal distinguishes itself through database-level transaction isolation for booking (addressing the concurrency gap identified in Section 2.5) and the integrated rule-based chatbot for query assistance.

[Figure 7.2: Comparison Chart — Insert radar/spider chart comparing Academic Portal vs Moodle vs Custom Portal across dimensions: Appointment Scheduling, Chatbot, Course Management, Policy Management, Cost, Customizability]

7.4 Limitations

The following limitations were identified during development and testing:

- Password Security: SHA-256 hashing is not suitable for production password storage. Migration to Argon2id is essential before institutional deployment.
- Chatbot Scope: The rule-based chatbot is limited to predefined intent patterns. It cannot understand paraphrased or complex natural language queries. An ML-based intent classifier would significantly improve user experience.
- Scalability: The current architecture has not been load-tested beyond 50 concurrent users. Deployment at institutional scale would require connection pooling configuration review, caching strategies, and potentially a CDN for static assets.
- Notification System: No email or push notification system is implemented. Users are not automatically notified of appointment bookings, cancellations, or policy updates.
- Audit Logging: The AuditService is partially implemented. Full audit trail coverage for all administrative operations requires completion.

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

The Academic Portal project has successfully achieved its primary objective: the design, development, and documentation of a full-stack web application that addresses the operational management needs of academic departments. The system delivers a role-based, transactional, and user-friendly platform for managing users, courses, appointments, feedback, policies, and user assistance.

The implementation demonstrates comprehensive application of software engineering principles across the complete development lifecycle. The requirements analysis phase produced well-defined functional and non-functional requirements grounded in identified real-world problems. The system design phase produced a coherent architecture with UML artifacts including use case, class, sequence, and activity diagrams, as well as a normalized relational database schema. The implementation phase delivered working, security-conscious code in C# with ASP.NET Core MVC, ADO.NET, and SQL Server.

A key technical contribution of this project is the transactional appointment booking algorithm using SQL Server transactions with RepeatableRead isolation level, which reliably prevents double-booking race conditions — a gap identified in the literature review of prior academic scheduling systems. The integrated rule-based chatbot provides a practical, low-cost user assistance mechanism that reduces repetitive administrative queries.

The testing phase documented and resolved critical bugs (particularly the concurrency issue BUG-01) and confirmed through unit, integration, and system tests that all functional requirements are met. Performance measurements confirm that all operations meet the defined non-functional performance requirements.

8.2 Key Achievements and Learning Outcomes

8.2.1 Technical Achievements

- Successfully implemented a complete MVC web application with three distinct user roles and role-specific interfaces.
- Designed and implemented a transactional booking engine using SQL Server transactions, demonstrating database concurrency control.
- Built a functional rule-based chatbot service integrated into the portal's user interface.
- Applied security best practices including parameterized queries, antiforgery tokens, and role-based authorization.
- Achieved all functional requirements with passing test results across unit, integration, and system test suites.

8.2.2 Academic Learning Outcomes

- Software Development Lifecycle: Practical experience in requirements analysis, system design, implementation, testing, and documentation.
- Database Design: Applied normalization principles to design a relational schema with appropriate constraints, foreign keys, and indexes.
- Web Application Development: Hands-on proficiency with ASP.NET Core MVC, Razor templating, Bootstrap, and jQuery.
- Technical Writing: Developed skills in producing formal technical documentation including UML diagrams, test case tables, and academic report writing.

8.3 Future Scope and Enhancements

The following enhancements are recommended for future development iterations:

8.3.1 Security Enhancements

41. Password Hashing Migration: Replace SHA-256 with Argon2id via the Konscious.Security.Cryptography library. Argon2id is the current recommended password hashing algorithm (OWASP 2024) due to its resistance to GPU-accelerated brute-force attacks through configurable memory and time costs.

42. Two-Factor Authentication (2FA): Implement TOTP-based 2FA using a library such as GoogleAuthenticator for administrative accounts to enhance security.
43. Content Security Policy (CSP): Add CSP headers to mitigate XSS risks beyond the current antiforgery token protection.

8.3.2 Feature Enhancements

44. Email Notification System: Integrate SMTP-based or SendGrid email notifications for appointment bookings, cancellations, and policy publications. This addresses a key user experience gap identified in the results analysis.
45. Calendar Integration: Implement two-way synchronization with Google Calendar and Microsoft Outlook using their respective APIs (Google Calendar API, Microsoft Graph API), enabling users to see appointments in their existing calendar applications.
46. ML-Based Chatbot: Replace the rule-based ChatbotService with an intent classification model trained on academic query datasets. A lightweight transformer model (DistilBERT fine-tuned on domain data) or integration with a commercial NLP API (such as Dialogflow or Microsoft LUIS) would substantially improve chatbot accuracy and coverage.
47. Payment Gateway: Integrate a payment gateway (Razorpay for Indian institutions or Stripe for international deployment) to support fee collection for courses or services within the portal.
48. Advanced Analytics Dashboard: Expand the admin dashboard with data visualizations (Chart.js) for enrollment trends, appointment utilization rates, feedback sentiment distributions, and policy access patterns.

8.3.3 Infrastructure Enhancements

49. Containerization: Package the application using Docker with a multi-stage Dockerfile, enabling consistent deployment across development, testing, and production environments.
50. Cloud Deployment: Deploy the containerized application to Azure App Service with Azure SQL Database, taking advantage of managed scaling, automated backups, and Azure Active Directory integration for enterprise SSO.

51. CI/CD Pipeline: Implement a GitHub Actions workflow for automated build, test, and deployment on code commits to the main branch, ensuring continuous integration and quality assurance.
52. Mobile Application: Develop a companion mobile application using .NET MAUI or React Native that consumes the existing API endpoints, providing native mobile access for students and faculty.

CHAPTER 9: REFERENCES

Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, technology, and applications. Machine Learning with Applications, 2, 100006. https://doi.org/10.1016/j.mlwa.2020.100006

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... & Thomas, D. (2001). Manifesto for agile software development. Agile Alliance. https://agilemanifesto.org

Blackboard Inc. (2023). Blackboard learn product overview. Anthology Inc. https://www.anthology.com/blackboard-learn

Bootstrap Team. (2023). Bootstrap 5 documentation. The Bootstrap Authors. https://getbootstrap.com/docs/5.3

Date, C. J. (2004). An introduction to database systems (8th ed.). Addison-Wesley.

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Ganss, B. (2023). HtmlSanitizer: A library for sanitizing HTML to avoid XSS attacks. GitHub. https://github.com/mganss/HtmlSanitizer

Kumar, R., & Sharma, P. (2022). Digital appointment management in Indian universities: Challenges and solutions. International Journal of Educational Technology, 14(3), 45–62.

Microsoft Corporation. (2024). ASP.NET Core documentation. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core

Microsoft Corporation. (2024). ADO.NET overview. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview

Microsoft Corporation. (2024). Microsoft.Data.SqlClient documentation. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/microsoft.data.sqlclient

Microsoft Corporation. (2024). SQL Server documentation. Microsoft Learn. https://learn.microsoft.com/en-us/sql/sql-server

Moodle Pty Ltd. (2024). Moodle statistics. Moodle.org. https://stats.moodle.org

OWASP Foundation. (2024). Password storage cheat sheet. OWASP. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (2024). SQL injection prevention cheat sheet. OWASP. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Patel, A., Singh, R., & Gupta, M. (2023). Design and implementation of an appointment management system for engineering colleges using PHP and MySQL. International Journal of Computer Science and Engineering, 11(2), 78–92.

Pressman, R. S., & Maxim, B. R. (2019). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 26, 1–9.

Rumbaugh, J., Jacobson, I., & Booch, G. (2004). The unified modeling language reference manual (2nd ed.). Addison-Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database system concepts (7th ed.). McGraw-Hill Education.

SweetAlert2 Contributors. (2024). SweetAlert2 documentation. https://sweetalert2.github.io

Winkler, R., & Soellner, M. (2018). Unleashing the potential of service chatbots: Recommendations for service management. Proceedings of the 51st Hawaii International Conference on System Sciences. https://doi.org/10.24251/HICSS.2018.347

CHAPTER 10: APPENDICES

Appendix A: Project File and Module Map

Path	Description
Acedemic Portal/Controllers/AccountController.cs	Authentication: Login, Logout, session management
Acedemic Portal/Controllers/DashboardController.cs	Role-specific dashboard views and metrics
Acedemic Portal/Controllers/AppointmentsController.cs	Appointment booking, listing, cancellation; GetAvailableSlots AJAX
Acedemic Portal/Controllers/ChatbotController.cs	Chatbot HTTP endpoint; delegates to ChatbotService
Acedemic Portal/Controllers/CoursesController.cs	Student-facing course browsing and enrollment
Acedemic Portal/Controllers/FeedbackController.cs	Feedback submission and listing
Acedemic Portal/Controllers/FacultyController.cs	Faculty profile and slot management
Acedemic Portal/Controllers/PoliciesController.cs	Policy browsing and search
Acedemic Portal/Controllers/Admin/*.cs	Admin CRUD controllers for users, courses, reports
Acedemic Portal/Services/DataAccessService.cs	Centralized ADO.NET data access layer
Acedemic Portal/Services/ChatbotService.cs	Rule-based chatbot intent detection and response generation
Acedemic Portal/Services/AuditService.cs	Administrative action audit logging
Acedemic Portal/Models/ViewModels/*.cs	Strongly-typed ViewModel classes for all views
Acedemic Portal/Views/**/*.cshtml	Razor view templates organized by controller
SQL/init_schema.sql	Complete database schema: CREATE TABLE

	statements
SQL/create_db_and_seed.ps1	PowerShell script for database creation and test data seeding
SQL/add_appointment_feedback.sql	Additional migration script for feedback schema additions

Appendix B: Key Database Scripts

B.1 Core Table Creation (Excerpt from SQL/init_schema.sql)

```
-- Users Table
CREATE TABLE Users (
    UserId    INT IDENTITY(1,1) PRIMARY KEY,
    Username  NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    FullName  NVARCHAR(255) NOT NULL,
    Email     NVARCHAR(255) NOT NULL UNIQUE,
    Role      NVARCHAR(20)  NOT NULL CHECK (Role IN ('Admin','Faculty','Student')),
    CreatedAt DATETIME DEFAULT GETDATE()
);

-- AppointmentSlots Table
CREATE TABLE AppointmentSlots (
    SlotId    INT IDENTITY(1,1) PRIMARY KEY,
    FacultyId INT NOT NULL REFERENCES Users(UserId),
    SlotDate  DATE NOT NULL,
    StartTime TIME NOT NULL,
    EndTime   TIME NOT NULL,
    IsAvailable BIT DEFAULT 1,
    CreatedAt DATETIME DEFAULT GETDATE()
);

-- Appointments Table
CREATE TABLE Appointments (
    AppointmentId INT IDENTITY(1,1) PRIMARY KEY,
    SlotId      INT NOT NULL REFERENCES AppointmentSlots(SlotId),
    StudentId   INT NOT NULL REFERENCES Users(UserId),
    Title       NVARCHAR(255) NOT NULL,
```

```
    Description NVARCHAR(MAX) NULL,
    Status      NVARCHAR(20) DEFAULT 'Scheduled',
    CreatedAt   DATETIME DEFAULT GETDATE()
);

-- Recommended indexes for performance
CREATE INDEX IX_AppointmentSlots_FacultyId ON AppointmentSlots(FacultyId);
CREATE INDEX IX_Appointments_StudentId ON Appointments(StudentId);
CREATE INDEX IX_Enrollments_StudentCourse ON Enrollments(StudentId, CourseId);
```

Appendix C: Installation Guide

53. Clone the repository from GitHub: git clone https://github.com/[your-username]/academic-portal.git
54. Ensure .NET 8 SDK is installed: verify by running dotnet --version in a terminal.
55. Restore NuGet packages: run dotnet restore from the project root directory.
56. Build the project: run dotnet build -p:UseAppHost=false to verify no compilation errors.
57. Create the database: execute SQL/init_schema.sql on your SQL Server instance, or run the PowerShell seeding script: .\SQL\create_db_and_seed.ps1
58. Configure the connection string: update the ConnectionStrings:DefaultConnection value in appsettings.json or set it via environment variable.
59. Run the application: execute dotnet run from the Acedemic Portal directory. The application will be available at https://localhost:5001.
60. Default admin credentials: Username: admin, Password: admin123 (change immediately after first login).

Appendix D: User Manual (Brief)

D.1 Admin User Guide

- User Management: Navigate to Admin > Users. Click "Add User" to create a new account. Select the role (Faculty/Student) and fill in required fields. Click Save.
- Course Management: Navigate to Admin > Courses. Click "Add Course", fill in title, description, assign a faculty member, and set credit hours. Toggle IsActive to control course visibility.

- Policy Management: Navigate to Policies > Manage. Click "Add Policy", enter title and content. Published policies are immediately visible to all users.

D.2 Faculty User Guide

- Managing Slots: Navigate to Appointments > My Slots. Click "Add Slot", select date and time range. Slots immediately appear in the student booking interface.
- Viewing Appointments: Navigate to Appointments > My Appointments to see all student bookings with details.

D.3 Student User Guide

- Course Enrollment: Navigate to Courses. Browse available courses and click "Enroll" on any course you wish to join. Enrollment status is shown on the course card.
- Booking Appointments: Navigate to Appointments > Book. Select a faculty member and date from the dropdowns. Available slots load automatically. Click "Book" on your preferred slot and fill in the appointment details.
- Using the Chatbot: Click the chatbot icon in the sidebar. Type your query (e.g., "What is the attendance policy?" or "Show available courses"). The chatbot will respond with relevant information and links.
- Submitting Feedback: Navigate to Feedback > Submit. Select a course, choose a star rating, and add comments. Click Submit to record your feedback.